Practical 10

Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition: Develop secure login and logout functionality using PHP sessions.Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session
timeout, and optional secure remember-me token.

A. Theory

1. Are sessions started and destroyed securely?

Answer:

Yes, PHP sessions are started using session_start(). After successful login, the session ID is regenerated using session_regenerate_id(true). During logout, session data is cleared, the session cookie is invalidated, and the session is destroyed.

https://www.w3schools.com/php/php_sessions.asp

2. Is password_verify() used correctly?

Answer:

Yes, password_verify() checks the entered password against the password hash stored in the database. This allows authentication without storing plain-text passwords.

https://www.w3schools.com/php/func_crypt_password_verify.asp

3.Is unauthorized dashboard access blocked?

Answer:

Yes, dashboard pages check whether the user has a valid login session before displaying protected content. If a user is not authenticated, they are redirected to the login page, preventing unauthorized access

https://www.geeksforgeeks.org/php/how-to-redirect-a-user-to-the-registration-if-he-has-not-logged-in/

4.Is session timeout implemented?

Answer:

Yes, a session timeout is implemented by tracking the user's last activity time. If the user remains inactive beyond the configured time limit, the session is invalidated and the user must log in again.

https://www.geeksforgeeks.org/php/php-sessions/

4.Are admin and student users redirected to appropriate dashboards?

Answer:

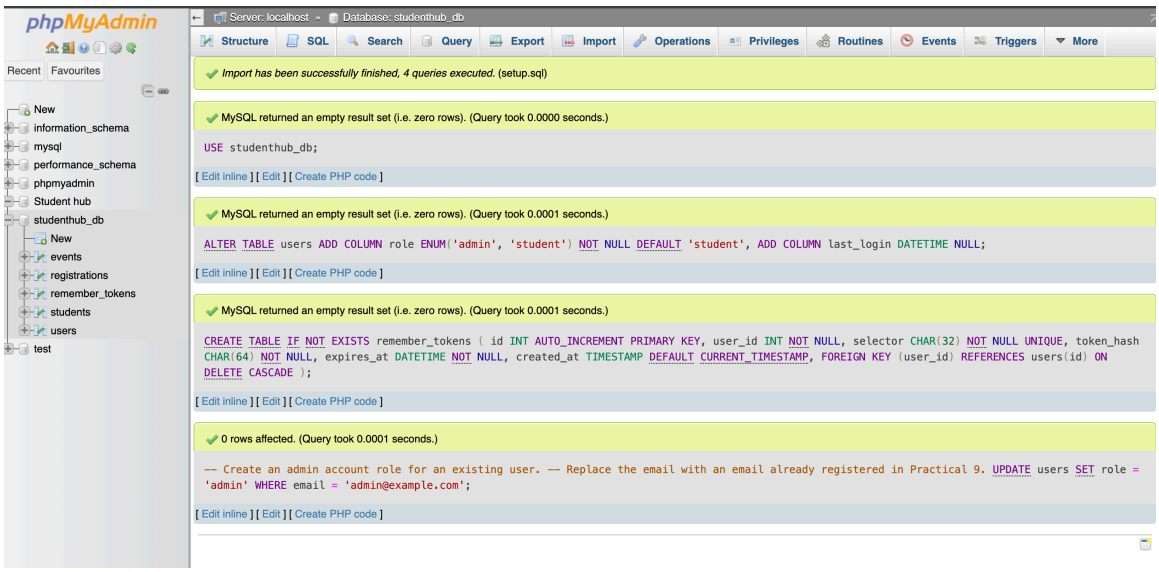
Yes, role-based access control checks the user's assigned role after successful authentication. Administrators are redirected to the admin dashboard, while students are redirected to the student dashboard. Each dashboard must also verify the user's role to prevent unauthorized access

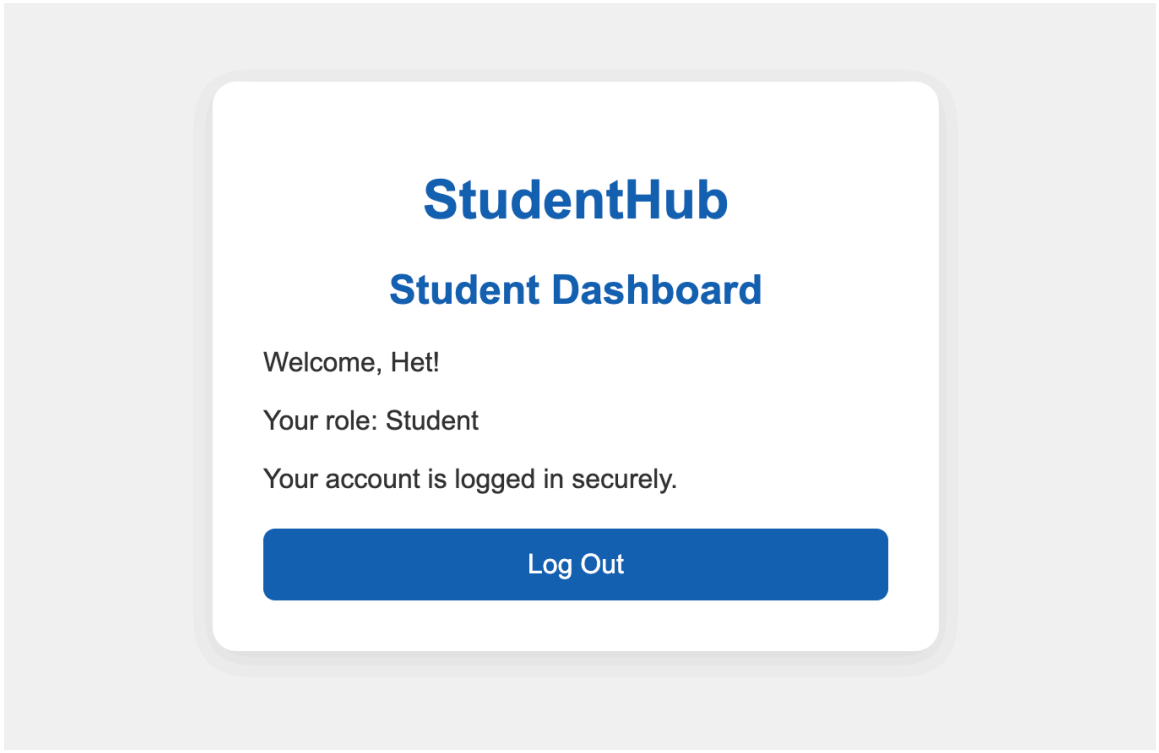
https://www.geeksforgeeks.org/ethical-hacking/role-based-access-control-1/

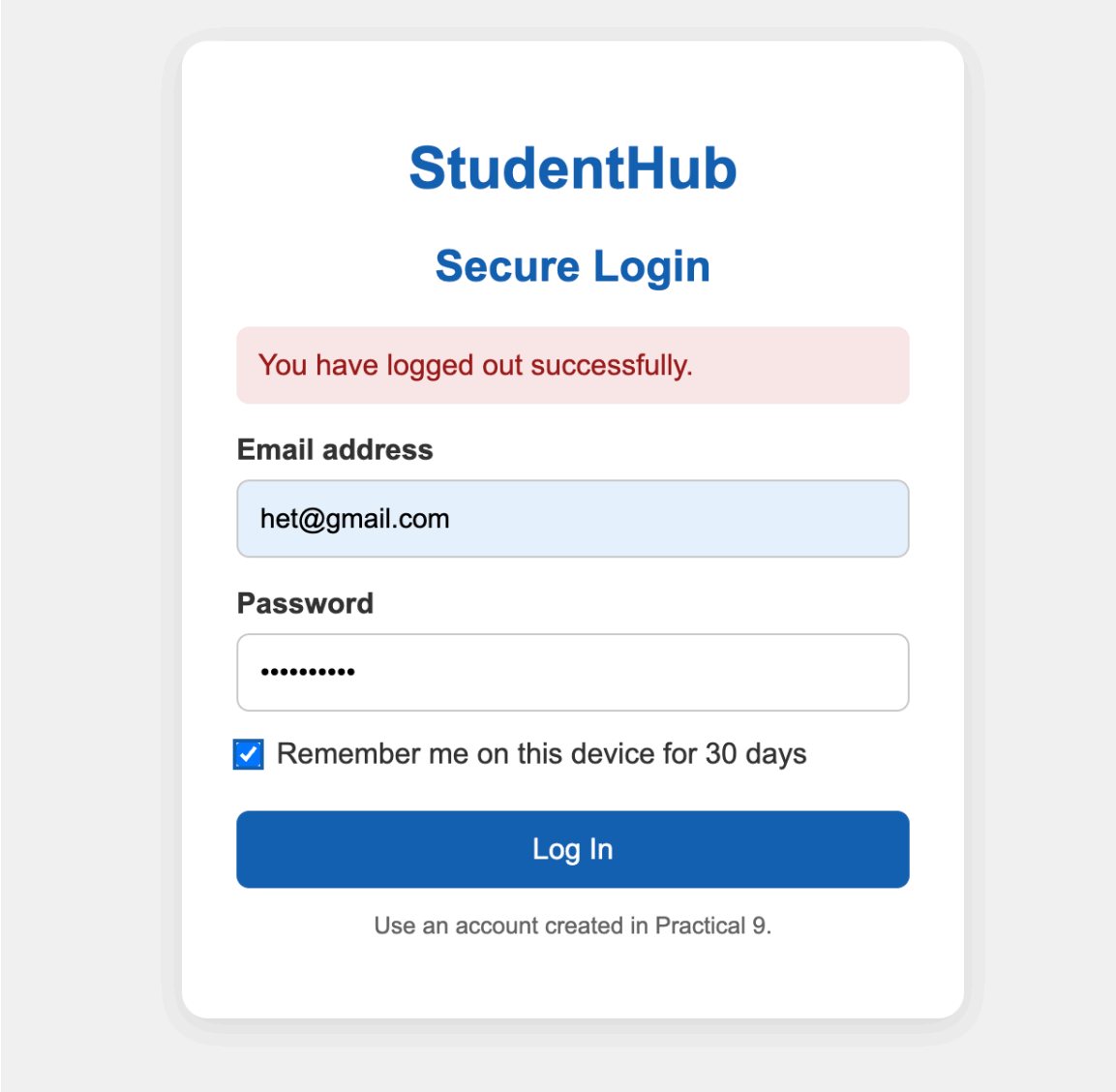
B. Result

Thus, secure login and logout functionality was implemented using PHP sessions and password verification. Session ID regeneration, dashboard access restrictions, session timeout, and role-based redirection help improve authentication security and prevent unauthorized access.

C. Evidence of Work (Screenshots)







D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

The main challenges faced were managing PHP sessions securely, verifying passwords using password_verify(), preventing unauthorized dashboard access, implementing session timeout, and redirecting admin and student users to their respective dashboards.

F. Learning Outcomes:

This practical helped in understanding secure authentication using PHP sessions, password verification, session ID regeneration, logout handling, access restrictions, session timeout, and role-based access control. It also introduced the importance of secure cookie management and remember-me tokens in developing reliable web authentication systems.

Faculty Remarks: